```
            }                                                          ❹
            var joiningDate = Convert.ToDateTime(                      ❺
            ➥ joiningDateClaim);                                      ❺
            if(context.User.HasClaim("Permission",                     ❺
            ➥ "View Roles") &&                                        ❺
                joiningDate > DateTime.MinValue &&                     ❺
                joiningDate < DateTime.Now.AddMonths(                  ❺
                ➥ Months))                                            ❺
            {                                                          ❺
                context.Succeed(this);                                 ❺
            }                                                          ❺
            return Task.CompletedTask;                                 ❻
        }
    }
}
```

❶ The class implements both the IAuthorizationRequirement and IAuthorizationHandler interfaces.

❷ The constructor takes an int as a parameter and ensures it is not a positive number.

❸ The HandleAsync method is implemented as required by the IAuthorization-Handler interface.

❹ The user is checked to see if they have a joiningDateClaim. If not, the handler is exited.

❺ The joining date is assessed to see if it exists and if its value is older than the age passed in.

❻ If the requirement is not satisfied, Task.CompletedTask is returned to satisfy the HandleAsync method signature.

The requirement is satisfied if it is marked as being successfully evaluated. This is achieved by calling the `Succeed` method of the `AuthorizationHandlerContext` class. This class also offers a `Fail` method, which you can call to ensure authorization is not successful. You would use this method if your handler allowed all users *except* those that meet the specified condition, for example. You use the `PolicyBuilder` to register the policy, passing in a suitable value for the `months` parameter.

Listing 10.20 Registering a custom requirement

```
builder.Services.AddAuthorization(options =>
{
    options.AddPolicy("ViewRolesPolicy", policyBuilder =>
```